

Day 18 - Lecture 18.7: Styling & Saving Plots

1. Overview & Built-in Style Sheets

Matplotlib comes with dozens of pre-configured style sheets that change fonts, colors, backgrounds, and grid styles in a single command.

Viewing and Applying Styles:

```
# List all available styles
print(plt.style.available)

# Apply a style globally
plt.style.use('ggplot')           # R's ggplot2 theme
plt.style.use('dark_background')  # Sleek dark mode
plt.style.use('default')          # Resets back to factory default
```

Comic Style with `xkcd()`:

Matplotlib includes a whimsical context manager that styles plots in the hand-drawn sketch style of the XKCD webcomic:

```
with plt.xkcd():
    plt.plot(x, y)
plt.show()
```

2. Preventing Clipping with `plt.tight_layout()`

When long axis labels or titles extend close to the figure boundary, calling:

```
plt.tight_layout()
```

automatically adjusts subplots, margins, and padding so all elements fit within the canvas cleanly.

3. Saving High-Resolution Figures (`plt.savefig()`)

To export your visualization for reports, papers, or websites:

```
plt.savefig(  
    "figure.png",          # File name and format (.png, .pdf, .svg, .jpg)  
    dpi=300,               # Dots Per Inch (resolution: 300 dpi is print standard)  
    bbox_inches="tight",   # Automatically trims excess white margins  
    transparent=False      # Whether background should be transparent  
)
```

[!CAUTION] Always call `plt.savefig()` **BEFORE** calling `plt.show()`. Calling `plt.show()` resets the current figure buffer, causing `plt.savefig()` afterwards to save a blank white image!

4. Code Implementation

```
import matplotlib.pyplot as plt

oscar_years = [2008, 2009, 2010, 2011, 2012]
oscar_revenue = [1005, 170, 427, 133, 232]
non_oscar_revenue = [378, 2788, 829, 185, 275]

# XKCD hand-drawn sketch style
with plt.xkcd():
    plt.figure(figsize=(8, 5))
    plt.plot(oscar_years, oscar_revenue, "o-b", label="Oscar")
    plt.plot(oscar_years, non_oscar_revenue, ">--g", label="Non-Oscar")
    plt.xlabel("Years")
    plt.ylabel("Revenue ($M)")
    plt.legend()
    plt.tight_layout()
    plt.savefig("movie_revenue_xkcd.png", dpi=150)
    plt.show()

# Reset to default
plt.style.use("default")
```

5. Key Takeaways

- For presentations, `dark_background` looks great on screens. For print papers, use `default` or `tableau-colorblind10` with high DPI (`dpi=300`).
- Always use `bbox_inches="tight"` to avoid cropped axis labels.